IERG4320/IEMS5726: Data Science in Practice

Lecture 3: Data Representation I

Prof. YAN Wenjing, Dr. Danny Yip
Department of Information Engineering

香港中文大學
The Chinese University of Hong Kong

Outline

1. Introduction to Data Preprocessing
2. Different Datasets
3. Data Imputation
4. Dealing with Categorical Variables
5. Duplication Removal
6. Data Normalization
7. Dealing with Large Numeric Data Matrix

All rights are reserved to Prof. YAN Wenjing, Dr. Danny YIP,
The Department of Information Engineering, CUHK

Introduction to Data Preprocessing

Data Preprocessing: Manipulation or dropping of data before it is used in order to ensure or enhance performance

Data Wrangling: A step before data preprocessing, or part of data preprocessing

- Very rarely is data easily accessible in a data science project for analysis.
- It is the process of transforming and mapping data from one "raw" data form into another format with the intent of making it more appropriate and valuable for a variety of downstream purposes such as analytics.
- It is the process of cleaning and unifying messy and complex data sets for easy access and analysis.

Steps for Data Wrangling

1. **Discovering**: Understand different datasets.
2. **Structuring**: Organize the raw data.
3. **Cleaning**: Catch data formats and remove outliers.
4. **Enriching**: Determine whether additional data can be added to easily.
5. **Validating**: Check data consistency, data quality, and data security.
6. **Publishing**: Prepare the dataset for downstream usage.

Name	Phone	Birth Date	State
John, Smith	445-881-4478	August 12, 1989	Maine
Jennifer Tal	+1-189-456-4513	11/12/1965	Tx
Gates, Bill	(876)546-8165	June 15, 72	Kansas
Alan Fitch	5493156648	2-6-1985	Oh
Jacob Alan	156-4896	January 3	Alabama

Name	Phone	Birth Date	State
John Smith	445-881-4478	1989-08-12	Maine
Jennifer Tal	189-456-4513	1965-11-12	Texas
Bill Gates	876-546-8165	1972-06-15	Kansas
Alan Fitch	549-315-6648	1985-02-06	Ohio

Why Data Preprocessing is Important?

1. **Accuracy**: To check whether the data entered is correct or not.
2. **Completeness**: To check whether the data is available or not recorded.
3. **Consistency**: To check whether the same data is kept in all the places that do or do not match.
4. **Timeliness**: The data should be updated correctly.
5. **Believability**: The data should be trustable.
6. **Interpretability**: The understandability of the data.

Goals of Data Preprocessing

1. **Data cleaning:** The process of fixing or removing incorrect, corrupted, incorrectly formatted, duplicate, or incomplete data within a dataset.
2. **Data integration:** The technical and business processes used to combine data from multiple sources to provide a unified, single view of the data.
3. **Data reduction:** The process of reducing the amount of capacity required to store data.
4. **Data transformation:** The process of changing the format, structure, or values of data.

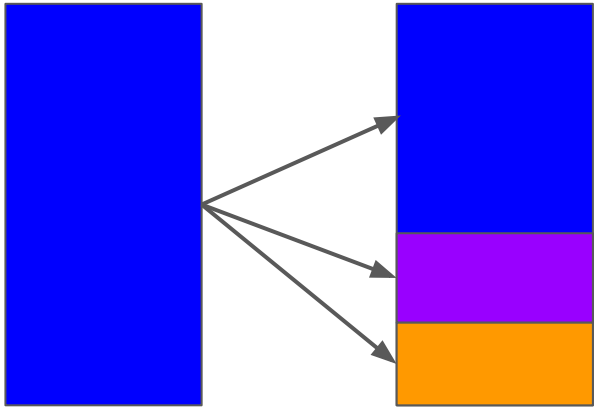
Steps for Data Preprocessing

1. Load the libraries and dataset
2. Perform data wrangling
3. Dealing with missing data or data imputation
4. Dealing with categorical variables
5. Remove duplication
6. Split the dataset into training, validation and testing set
7. Perform data normalization
8. (Optional nowaday) Perform feature selection

Different Datasets

Training, Validation and Testing set

Full data set



Training data set: fits the parameters for each model.

Validation data set: provides an unbiased evaluation of a fitted model.

Testing data set: provides an unbiased evaluation of a final fitted model.

```
from sklearn.model_selection import train_test_split
X_1,X_test,y_1,y_test = train_test_split(X, y, test_size=0.2, random_state=0)
X_train,X_val,y_train,y_val = train_test_split(X_1, y_1, test_size=0.2, random_state=0)
```

Cross Validation - Reasons for setting 3 data sets

- Cross validation estimates how well the model you have learned from the training data is going to perform on future as-yet-unseen data.
- For example, 5-fold cross validation
  - There are 5 iterations.
  - 1-fold is used as validation, 4-fold are used as training.
  - The reported CV metrics are based on the stack of the predictions of the 5 folds.

Color code:

- **Blue - Training set**
- **Purple - Validation set**
- **Orange - Testing set**

Fold\Iter	1	2	3	4	5
1					
2					
3					
4					
5					
6					

Example: Cross Validation and Leave-one-out

Which model do you choose?

Models	10-fold CV accuracy	Testing accuracy	Your choice?
M1	90%	60%	Best CV performance, but poor testing performance
M2	66%	90%	Best testing performance, but poor CV performance
M3	80%	88%	Testing performance is slightly better than CV performance
M4	88%	80%	CV performance is slightly better than testing performance

Data Imputation

Data imputation is the process of replacing missing values in a dataset with substituted, statistically plausible values. It ensures that incomplete datasets remain usable for analysis or machine learning without discarding valuable information.

Example: Sources of Missingness

1. **Data Collection Errors:** A survey was conducted and values were just randomly missed when being entered in the computer.
2. **Intentional Missingness:**
   - A respondent chooses not to respond to a question like “Have you ever recreationally used opioids?”.
   - You decide to start collecting a new variable (due to new actions: like a pandemic) partway through the data collection of a study.
3. **Systematic Issues:** You want to measure the speed of meteors, and some observations are just “too quick” to be measured properly

Type of Missingness

1. **Missing Completely at Random (MCAR)**: the probability of missingness in a variable is the same for all units. Like randomly poking holes in a data set.
   - E.g.: A coin is flipped to determine whether an entry is removed.
2. **Missing at Random (MAR)**: the probability of missingness in a variable depends only on available information (in other predictors).
   - E.g.: Men and women respond to the question “have you ever felt harassed at work?”
3. **Missing Not at Random (MNAR)**: the probability of missingness depends on information that has not been recorded and this information also predicts the missing values.
   - E.g. Patients drop out of a study because they experience some really bad side effect that was not measured. Or cheaters are less likely to respond when asked if you've ever cheated.

Dealing with Missing Data

1. Use deletion methods to eliminate missing data
   - Listwise deletion: drop whole row or whole column

```
df.drop([0,1])                        # Drop row by index
df.drop(columns=['A','B'], axis=1) # Drop column by labels
df.dropna()                           # Drop all NaN by row
```

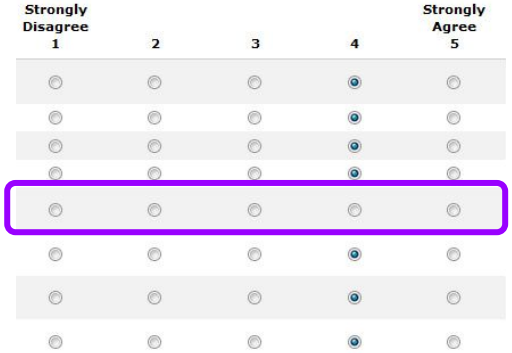
   - Pairwise deletion: row / column counts depend on non-missing values of particular row / column
2. Recover the values by contacting the participants

Data Imputation

Imputation is replacing missing values with substitute values.

1. **Educated guessing**: A guess based on knowledge or Experience.

   If you want to get all modal values, you may refer to `statistics.multimode(vector)`



What is educated guessing?

2. **Average imputation**: Use average of the response.
   - E.g.
   ```
   df.replace({'D':np.nan}, np.mean(df['D']))
   ```
3. **Common-point imputation**: Use median or mode of the response.
   - E.g.
   ```
   df.replace({'B':np.nan}, np.nanmedian(df['B']))
   df.replace({'D':np.nan}, stats.mode(df['D'])[0][0])
   ```

Regression Substitution

4. **Regression substitution**: Use multiple regression analysis to estimate value.

```
from sklearn import linear_model
df_train = df.dropna()
X = df_train.drop(columns=['D'])
regr = linear_model.LinearRegression()
regr.fit(X, df_train['D'])
regr.predict(df.drop(columns=['D']))
```

- This is
  - It may create **extra dependency** among the variables
  - It may cause drawback with the subsequent analysis

Hot Deck Imputation

5. **Hot deck imputation**: Replace with randomly selected values from same column.
   - E.g. `df.ffill(inplace=True)`
   - `ffill` that uses **last observation carried forward (LOCF)** Hot Code Imputation

```
data = {
  'Age': [25, 30, np.nan, 35, 40, np.nan, 28],
  'Gender': ['M', 'F', 'M', 'F', 'M', 'F', 'M'],
  'Income': [50000, 60000, np.nan, 70000, np.nan, 55000, 52000]
}
df = pd.DataFrame(data)
print(df)
df.ffill(inplace=True)
print(df)
```

	Age	Gender	Income
0	25.0	M	50000.0
1	30.0	F	60000.0
2	NaN	M	NaN
3	35.0	F	70000.0
4	40.0	M	NaN
5	NaN	F	55000.0
6	28.0	M	52000.0

	Age	Gender	Income
0	25.0	M	50000.0
1	30.0	F	60000.0
2	30.0	M	60000.0
3	35.0	F	70000.0
4	40.0	M	70000.0
5	40.0	F	55000.0
6	28.0	M	52000.0

Multiple Imputation

6. **Multiple imputation:** Allow for the uncertainty about the missing data by creating several different plausible imputed data sets and appropriately combining results obtained from each of them.

There are two major approaches:

1. **Multivariate normal:** Assuming imputation model, joint distribution of all variables, forms a multivariate normal.
2. **Imputation by chained equations:** Using conditional density (Bayesian) of a variable given other variables.

Steps:

1. Drop all row / column having all missing values
2. Use hotdeck approach to fill all missing values
3. For each variable, ignore the hotdeck substitute, construct a regression model
4. Analyze the variances (ANOVA) of each imputed dataset independently with the regression models

Example: Multiple Imputation

```python
import numpy as np
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer
imp = IterativeImputer(max_iter=10, random_state=0)
imp.fit([[1, 2], [3, 6], [4, 8], [np.nan, 3], [7, np.nan]])
X_test = [[np.nan, 2], [6, np.nan], [np.nan, 6]]
# the model learns that the second feature is double the first
print(np.round(imp.transform(X_test)))
```

```
IterativeImputer(random_state=0)
[[ 1.  2.]
 [ 6. 12.]
 [ 3.  6.]]
```

Reference: https://scikit-learn.org/stable/modules/impute.html

Different `sklearn`'s Impute Model

1. `sklearn.impute.SimpleImputer`: perform mean, median, mode,or any specific value to impute
2. `sklearn.impute.IterativeImputer`: perform multivariate imputation that estimates each predictor from all the others (usersupplied model/estimator).
3. `sklearn.impute.KNNImputer`: perform imputation automatically from a k-NN model from all the other predictors.
4. `sklearn.impute.MissingIndicator`: to create binary indicator(s) for where the missing values lie.

Guidelines for Missing Values, Harrell 2001

1. **(Low Missingness) Proportion of missings ≤ 0.05**: Imputation does not matter too much
   - Continuous variables - Median imputation
   - Categorical variables - Mode imputation
2. **(Moderate Missingness) Proportion of missings between 0.05 and 0.15**:
   - If predictor is unrelated to other variables, similar to previous situation.
   - If predictor is correlated with other variables, develop the multiple imputation model described previous and predict the missing values, then, adjust predictions based on variance
3. **(High Missingness) Proportion of missings between 0.15 and 0.3**: Multiple imputation
4. **(Very High Missingness) Proportion of missings > 0.3**: May drop variables

Harrell Jr FE, Regression Modeling Strategies with Applications to Linear Models, Logistic Regression and Survival Analysis. New York, Springer-Verlag:p49 (2001).

Dealing with Categorical Variables

Handling Categorical Data - Convert to Number

1. **Label encoder**: Transform non-numerical labels to categorical/nominal variables (e.g. sex: male → 1, female → 0)

```
from sklearn import preprocessing
label_encoder = preprocessing.LabelEncoder()
df['A']= label_encoder.fit_transform(df['A'])
df['A'].unique()
```

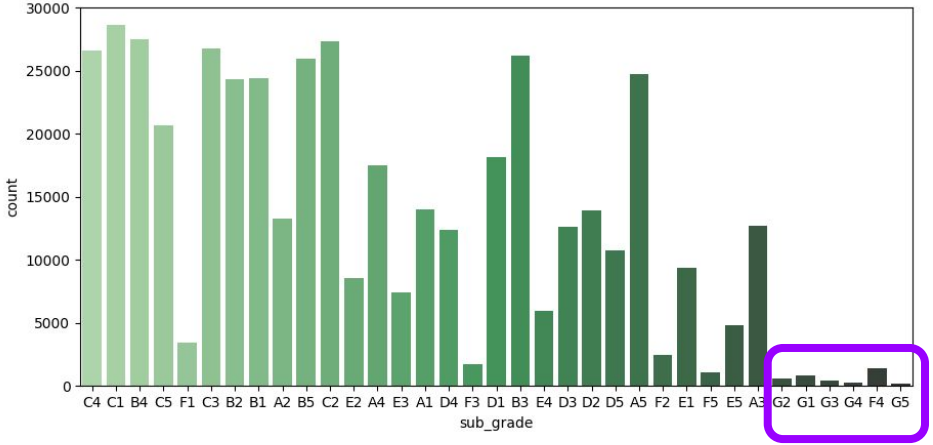
2. **Convert numeric bins to number**: Transform continuous variables to the label of bins (e.g. age: 0-17 → 0, 18-64 → 1, 65-inf → 2)

```
bins = [0, 17, 64, np.inf]
names = [0, 1, 2]
df['ageRange'] = pd.cut(df['age'], bins, labels=names)
```

Handling Categorical Data - Combine Levels

1. **Use business logic**: Combine similar levels into similar groups based on domain specific knowledge (e.g. combine zip code and district into 1 variable)
2. **Use frequency of response rate**: Combine variables with very low frequencies, or similar response rate (e.g. combine some subtype of cell compartment if their abundance is less than 5%)

- Later, in deep learning, there are more formal terms defined, which are **one-shot learning** and **few-shot learning**



Combine

Python implementations: https://www.datacamp.com/community/tutorials/categorical-data

Encoding Class Labels for Classification Problems

- Suppose we have 3 classes, which are "**red**", "**green**" and "**blue**".

1. **Ordinal encoding**: Variable comprises a finite set of discrete values with a ranked ordering between values.
   - "**red**" is **1**, "**green**" is **2**, and "**blue**" is **3**

```
df.name = pd.Categorical(df['name'])
df['code'] = df.name.cat.codes
```

2. **One-hot encoding**: The integer encoded variable is removed and one new binary variable is added for each unique integer value in the variable.
   - "**red**" is {**1**, 0, 0}, "**green**" is {0, **1**, 0}, and "**blue**" is {0, 0, **1**}
3. **Dummy variable encoding**: It reduces the redundancy from one-hot encoding.
   - "**red**" is {**1**, 0}, "**green**" is {0, **1**}, and "**blue**" is {0, 0}

- Note: These can convert categorical input variables into continuous variables.

Reference: https://machinelearningmastery.com/one-hot-encoding-for-categorical-data/
https://machinelearningmastery.com/how-to-one-hot-encode-sequence-data-in-python/#:~:text=A%20one%20hot%20encoding%20is,is%20marked%20with%20a%201.

Duplication Removal

Remove Duplication

- To ensure that the data is accurate and reliable for subsequent analysis

1. Drop duplicated rows

```
from pandas import pd
data = df.drop_duplicates()
```

2. Filter constant columns

```
data = df.loc[:, (df != df.iloc[0]).any()]
```

Remove Duplication

- To ensure that the data is accurate and reliable for subsequent analysis

3. Filter highly correlated rows or columns

```
def correlation(dataset, threshold):
  col_corr = set()
  corr_matrix = dataset.corr()
  for i in range(0, len(corr_matrix.columns)):
    for j in range(i):
      if abs(corr_matrix.iloc[i,j]) > threshold:
        colname = corr_matrix.columns[i]
        col_corr.add(colname)
  return col_corr
```

Data Normalization

Feature Scaling

- Since the range of values of raw data varies widely, in some machine learning algorithms, objective functions will not work properly without normalization.

1. Rescaling (min-max normalization)

$$x' = a + \frac{(x - \min(\vec{x}))(b-a)}{\max(\vec{x}) - \min(\vec{x})}$$

```
from sklearn import preprocessing
min_max_scaler = preprocessing.MinMaxScaler()
X_train_minmax = min_max_scaler.fit_transform(X_train)
```

2. Mean normalization

$$x' = \frac{x - \text{mean}(\vec{x})}{\max(\vec{x}) - \min(\vec{x})}$$

```
df_means = df.mean(axis=0)
min_max_range = df.max(axis=0) - df.min(axis=0)
df_scaled_mean = (df - df_means) / min_max_range
```

Feature Scaling

3. Standardization (z-score normalization)

$$x' = \frac{x - \text{mean}(\vec{x})}{\sigma}$$

```
from sklearn.preprocessing import StandardScaler
ss = StandardScaler()
df_scaled = ss.fit_transform(df)
```

4. Scaling to unit length

```
df/np.sqrt(np.square(df).sum(axis=0))
```

$$x' = \frac{x}{|\vec{x}|}$$

Dealing with Large Numeric Data Matrix

Feature Selection

- What happen if you open a spreadsheet with 1 million $\times$ 1 million matrix?
- If the ML algorithm takes O(nm) to train the model, how long will the ML algorithm take to complete training a model?
- **Feature Selection** is a process that chooses an optimal subset of features according to a certain criterion.

1. Removing irrelevant data.
2. Increasing predictive accuracy of learned models.
3. Reducing the cost of the data.
4. Improving learning efficiency, such as reducing storage requirements and computational cost.
5. Reducing the complexity of the resulting model description, improving the understanding of the data and the model.

Feature Selection

Two main approaches for feature selection

1. Selecting representative columns and/or rows
   - Simply picking the representative columns and/or rows without combining them, or resizing them
   - The results of the analysis can be directly associated or related with the original variables
2. Dimensionality reduction
   - Two major algorithms include principal component analysis (PCA) and linear discriminant analysis (LDA)

Perspectives of Selecting Columns and/or Rows

1. Searching for the best subset of features
   → Different searching strategies

2. Criteria for evaluating different subsets
   → Different searching criteria

3. Principle for selecting, adding, removing or changing new features during the search
   → Different searching directions

Direction and Strategies in the Search

Search Direction

1. Sequential Forward Generation (SFG)
2. Sequential Backward Generation (SBG)
3. Bidirectional Generation (BG)
4. Random Generation (RG)

Search Strategies

1. Exhaustive Search
2. Heuristic Search
3. Nondeterministic Search

Selection Criteria in the Search

1. **Information Measure**: Information serves to measure the uncertainty of the receiver when she/he receives a message
   - Shannon's Entropy $\sum_i P(c_i) \log_2 P(c_i)$
   - Information gain $IG(A) = I(D) - \sum_{j=1}^{p} \frac{|D_j|}{|D|} I(D_j^A)$

2. **Distance Measure**: Measures of separability, discrimination or divergence measures . The most typical is derived from distance between the class conditional density functions

	Mathematical form
Euclidean distance	$D_e = \left\{\sum_{i=1}^{m} (x_i - y_i)^2\right\}^{\frac{1}{2}}$
City-block distance	$D_{cb} = \sum_{i=1}^{m} \lvert x_i - y_i \rvert$
Cebyshev distance	$D_{ch} = \max_i \lvert x_i - y_i \rvert$
Minkowski distance of order m	$D_M = \left\{\sum_{i=1}^{m} (x_i - y_i)^m\right\}^{\frac{1}{m}}$
Quadratic distance Q, positive definite	$D_q = \sum_{i=1}^{m} \sum_{j=1}^{m} (x_i - y_i) Q_{ij} (x_j - y_j)$
Canberra distance	$D_{ca} = \sum_{i=1}^{m} \frac{\lvert x_i - y_i \rvert}{x_i + y_i}$
Angular separation	$D_{as} = \frac{\sum_{i=1}^{m} x_i \cdot y_i}{\left[\sum_{i=1}^{m} x_i^2 \sum_{i=1}^{m} y_i^2\right]^{\frac{1}{2}}}$

Selection Criteria in the Search

3. **Dependence Measures**: Also known as correlation or association, how strongly two variables are correlated, e.g. Pearson correlation coefficient

$$\rho(X, Y) = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\left[\sum_i (x_i - \bar{x})^2 \sum_i (y_i - \bar{y})^2\right]^{\frac{1}{2}}}$$

4. **Consistency Measures**: To find a minimum number of features that separate classes as the full set of features can, e.g. P(C|FullSet) = P(C|SubSet).
5. **Accuracy Measures**: Among various possible subsets of features, the subset which yields the best predictive accuracy is chosen

	Mathematical form
Accuracy	$\frac{tp+fp}{tp+tn+fp+fn}$
Error rate	$1-$ Accuracy
Chi-squared	$\frac{n(fp \times fn - tp \times tn)^2}{(tp+fp)(tp+fn)(fp+tn)(tn+fn)}$
Information gain	$e(tp+fn, fp+tn) - \frac{(tp+fp)e(tp,fp)+(tn+fn)e(fn,tn)}{tp+fp+tn+fn}$ where $e(x, y) = -\frac{x}{x+y}\log_2\frac{x}{x+y} - \frac{y}{x+y}\log_2\frac{y}{x+y}$
Odds ratio	$\frac{tpr}{1-tpr} \Big/ \frac{fpr}{1-fpr} = \frac{tp \times tn}{fp \times fn}$
Probability ratio	$\frac{tpr}{fpr}$

Example: Wrapper Method

```python
# Greedy search algorithms
# Utilise a specific classifier to select the optimal set of features.
# Sequential feature selection algorithms add or remove one feature at the time based
# on the classifier performances until a feature subset of the desired size k is
# reached, or any other desired criteria is met.
from mlxtend.feature_selection import SequentialFeatureSelector as SFS
import warnings
warnings.filterwarnings('ignore')

sfs1 = SFS(RandomForestClassifier(n_jobs=8),
          k_features=5,
          forward=True,
          floating=False,
          verbose=2,
          scoring='roc_auc',
          cv=3)
sfs1 = sfs1.fit(X_train.values, y_train)
selected_feat = X_train.columns[list(sfs1.k_feature_idx_)]
```

Reference:
https://github.com/codingnest/FeatureSelection/blob/master/Data%20Science%20Lifecycle%20-%20Feature%20Selection%20(Filter%2C%20Wrapper%2C%20Embedded%20and%20Hybrid%20Methods).ipynb

Example: Embedded Method - LASSO

```python
# Regularization consists in adding a penalty on the different parameters of the model
# to reduce the freedom of the model.
# Example: L1 regularization (Lasso)
from sklearn.linear_model import Lasso, LogisticRegression
from sklearn.feature_selection import SelectFromModel
from sklearn.preprocessing import StandardScaler

scalar = StandardScaler()
scalar.fit(X_train.values)

sel_ = SelectFromModel(LogisticRegression(C=1, penalty='l1'))
sel_.fit(scalar.transform(X_train.values), y_train)
sel_.get_support()
selected_feat = X_train.columns[sel_.get_support()]

X_train = sel_.transform(X_train)
X_test = sel_.transform(X_test)

```

Reference:
https://github.com/codingnest/FeatureSelection/blob/master/Data%20Science%20Lifecycle%20-%20Feature%20Selection%20(Filter%2C%20Wrapper%2C%20Embedded%20and%20Hybrid%20Methods).ipynb

Dimensionality Reduction

- In addition to selecting representative columns and/or rows, another group of researchers use **PCA** or **LDA** to reduce the dimension of the dataset
- Just select some important features from the whole dataset
- Run the ML algorithms with a subset of features, but covering most variances of the whole dataset.

Principal Component Analysis (PCA)

- PCA is an orthogonal projection or transformation of the data into a (possibly lower dimensional) subspace so that the variance of the projected data is maximized.
- Given data, $\{x_1, \ldots, x_m\}$, compute the principal vectors as:

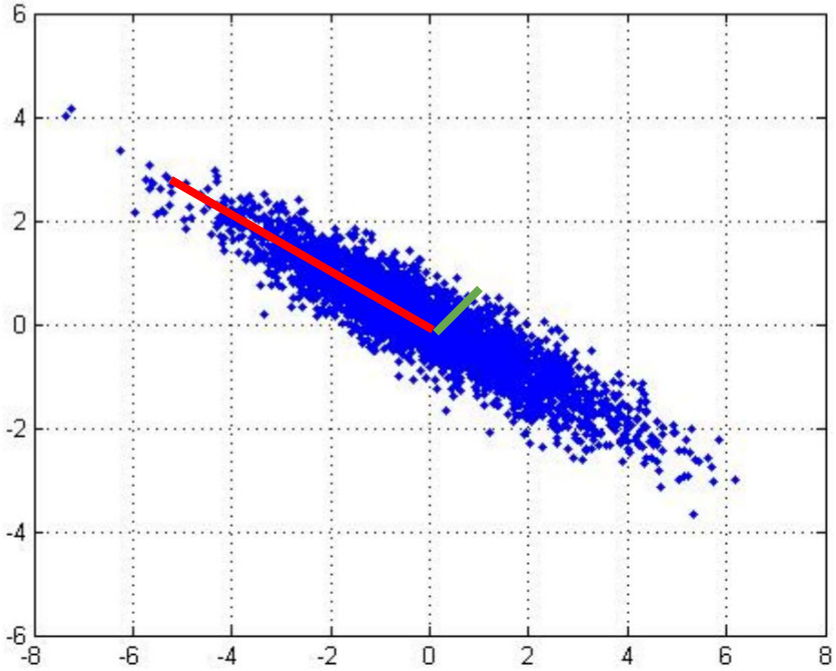
$$w_1 = \text{argmax}_{|w|=1} \frac{1}{m} \sum_i (w^T x_i)^2$$

- We can maximize the variance of the projection of x by:

$$w_k = \text{argmax}_{|w|=1} \frac{1}{m} \sum_i [w^T \underbrace{(x_i - \sum_j w_j w_j^T x_i)}_{\text{x' PCA reconstruction}}]^2$$

Visually: Principal Component Analysis (PCA)

- First PC is **red**
- Second PC is **green**
- Main idea: To maximize the variance of the projection in the residual subspace



Principal Component Analysis (PCA)

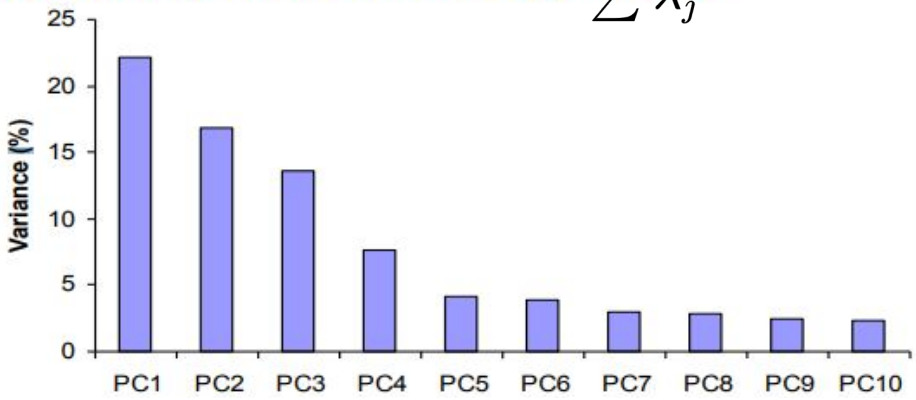
- Observation 1: PCA basis vectors = eigenvectors
- Observation 2: Larger eigenvalues → More important eigenvectors

- If you forget eigenvectors and eigenvalues, consider $S - \lambda I = 0$,
- Eigenvalues are the λ_i satisfying the solution to the equation.
- Eigenvectors are the corresponding solutions v_i, and all eigenvectors are linearly independent.

Variance explained and PCs

- "Variance" means summative variance or multivariate variability or overall variability or total variability.

$$\text{variance}_i = \frac{\lambda_i}{\sum \lambda_j}$$



- Pick the top PCs such that it accounts for a certain percentage of variance.

Reference: https://www.cs.cmu.edu/~mgormley/courses/10701-f16/slides/lecture14-pca.pdf

Two interpretations for PCA

1. Maximum variance direction
   - First PC captures maximum variance of the data.
2. Minimum reconstruction error
   - First PC yields minimum MSE reconstruction.

Remark:

- Another method, single value decomposition (SVD), is the square root of PCA.
- SVD: diag(σ_1, …, σ_m) such that σ_k = sqrt(λ_k)

Example: PCA for Dimension Reduction

```python
from sklearn import datasets
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
digits = datasets.load_digits()
# to remove the outliers and scale the data by making the mean of
# the data 0 and standard deviation as 1.
X = StandardScaler().fit_transform(digits.data)
# 0.85 as the percentage of feature in final dataset
pca = PCA(n_components=0.85, whiten=True)
X_pca = pca.fit_transform(X)
print(X_pca)
```

```
[[ 0.70631939 -0.39512814 -1.73816236 ...  0.60320435 -0.94455291 -0.60204272]
 [ 0.21732591  0.38276482  1.72878893 ... -0.56722002  0.61131544  1.02457999]
 [ 0.4804351  -0.13130437  1.33172761 ... -1.51284419 -0.48470912 -0.52826811]
 ...
 [ 0.37732433 -0.0612296   1.0879821  ...  0.04925597  0.29271531 -0.33891255]
 [ 0.39705007 -0.15768102 -1.08160094 ...  1.31785641  0.38883981 -1.21854835]
 [-0.46407544 -0.92213976  0.12493334 ... -1.27242756 -0.34190284 -1.17852306]]
```

Reference: https://www.projectpro.io/recipes/reduce-dimentionality-using-pca-in-python

Linear Discriminant Analysis (LDA) when p = 1

- LDA models the distribution of the predictors X given the K different categories that Y takes on, P(X = x | Y = k).
- By Bayes' Theorem, $P(Y=k|X=x) = \frac{f_k(x)P(Y=k)}{\sum_j f_j(x)P(Y=j)}$
- LDA has the simplest form, one feature / predictor.
- Assume f ~ N(μ, σ), based on the results of maximum log-likelihood estimation,

$$\hat{\mu}_k = \frac{1}{n_k}\sum_{i:y_i=k} x_i \text{ and } \hat{\sigma}^2 = \frac{1}{n-K}\sum_{k,i:y_i=k}(x_i - \hat{\mu}_k)^2$$

- Hence, we can estimate $P(Y=k) = \frac{n_k}{n}$
- After running LDA, we will have K-1 features.

Reference: https://harvard-iacs.github.io/2018-CS109A/lectures/lecture-14/presentation/lecture14_discriminant_analysis.pdf

Linear Discriminant Analysis (LDA) when p > 1

- Suppose the joint PDF of multivariate normal distribution (MVN) is,

$$\vec{X} \sim \mathrm{MVN}(\vec{\mu}, \sum) \quad f(\vec{x}) = \frac{1}{2\pi^{p/2} |\sum|^{0.5}} e^{-0.5(\vec{x}-\vec{\mu})^T \sum^{-1} (\vec{x}-\vec{\mu})}$$

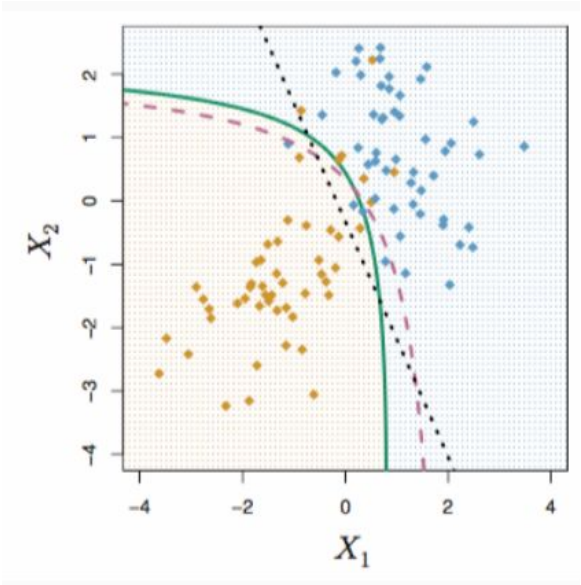
- Rewriting the posterior,

$$P(Y=k|\vec{X}=\vec{x}) = \frac{P(Y=k) \frac{1}{2\pi^{p/2} |\sum|^{0.5}} e^{-0.5(\vec{x}-\vec{\mu_k})^T \sum^{-1} (\vec{x}-\vec{\mu_k})}}{\sum_j P(Y=j) \frac{1}{2\pi^{p/2} |\sum|^{0.5}} e^{-0.5(\vec{x}-\vec{\mu_j})^T \sum^{-1} (\vec{x}-\vec{\mu_j})}}$$

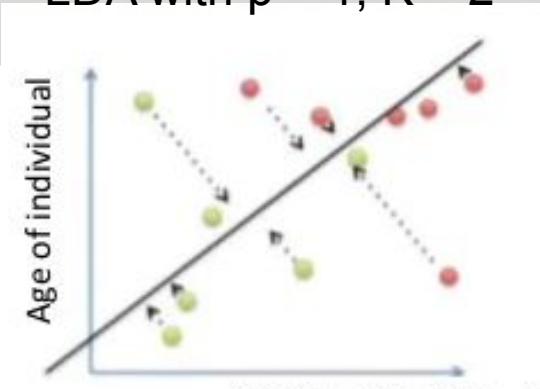
Some Visualizations of LDA

Dotted line: LDA with p = 2, K = 2

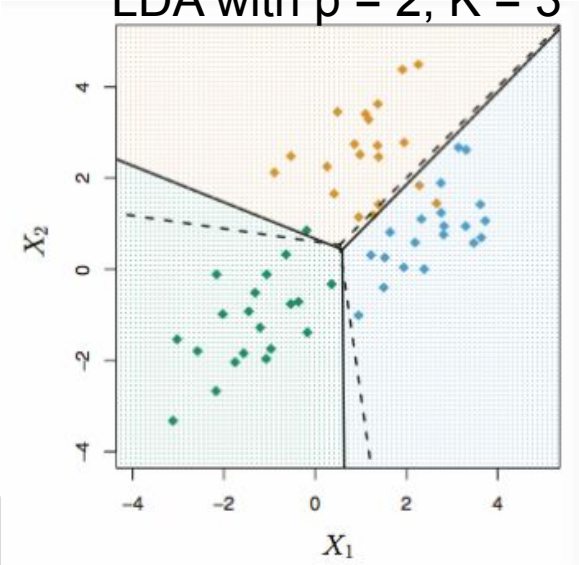
Solid line: QDA with p = 2, K = 2



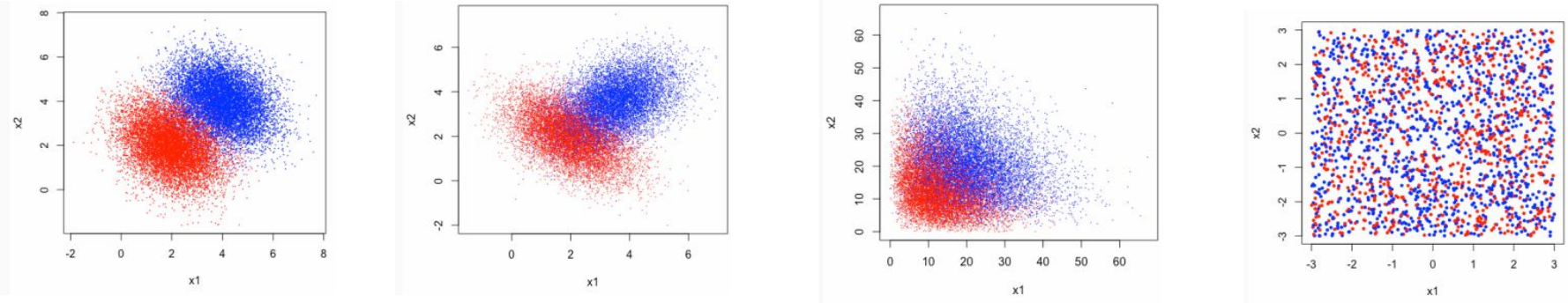
LDA with p = 1, K = 2



LDA with p = 2, K = 3



Which method, LDA or QDA, is better for separating two classes?



- QDA outperforms LDA if the covariances are not the same in the groups.

Example: LDA for Dimension Reduction

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
# Load dataset
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data"
names = ['sepal-length', 'sepal-width', 'petal-length', 'petal-width', 'Class']
dataset = pd.read_csv(url, names=names)
X = dataset.iloc[:, 0:4].values
y = dataset.iloc[:, 4].values
# divide the dataset into training and testing set
X_train,X_test,y_train,y_test = train_test_split(X, y, test_size=0.2, random_state=0)
# scale the data, similar to PCA
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
# LDA takes the output into account while selecting the LDs
lda = LDA(n_components=1)
X_train = lda.fit_transform(X_train, y_train)
X_test = lda.transform(X_test)
```

Reference: https://stackabuse.com/implementing-lda-in-python-with-scikit-learn/

Remarks for PCA and LDA

1. PCA seeks directions that are efficient for representing data, while LDA seeks directions that are efficient for discriminating data.
2. PCA is an unsupervised method, while LDA is a supervised method, using the class label in data preprocessing.
3. PCA returns the number of PCs up to min(number of columns, number of rows), while the number of LDs up to the number of class label - 1.
4. PCA maximizes the sum of variance explained, while LDA maximizes the class separation among the data points.

Other Data Reduction Skills

1. **t-Distributed Stochastic Neighbor Embedding:** Preserves local pairwise distances (great for visualization).
2. **Uniform Manifold Approximation and Projection**: Balances local/global structure (faster alternative to t-SNE).
3. **Singular Value Decomposition**: Generalization of PCA for sparse/dense matrices.
4. … etc

Drawbacks of Feature Selection

- The resulted subsets of many models of FS are **strongly dependent** on the training set size.
- It is not true that a large dimensionality input can always be reduced to a small subset of features because the objective feature is actually related with many input features and the **removal of any of them will seriously affect the learning performance**.
- A backward removal strategy is very slow when working with large-scale data sets. This is because in the firsts stages of the algorithm, it has to make decisions funded on huge quantities of data.
- In some cases, the feature selection outcome will still be left with a relatively large number of relevant features which even inhibit the use of complex learning methods.

What’s Next?

- We have learnt how to preprocess and handle the numeric data.
- The Python implementation for the feature selection on the numeric data.
- How about (i) **natural language** (text), (ii) **images**, (iii) **audio** and (iv) multimedia?